[CS 4731]Paper 3

1. ABSTRACT

This paper covers a genetic algorithm that is used to discover target grammars from given datasets of strings. Three experiments were run, with five trials per experiment. Each experiment aimed to discover target grammars with unique structures. Each experiment resulted in at least one semantic equivalent to the target grammar.

2. INTRODUCTION

2.1 Problem

This algorithm is a practice in grammatical evolution. The algorithm is attempting to find a target grammar, or semantic equivalent, through evolutionary techniques.

2.2 Search Space Reductions

To limit the scope of the search, the search space is constrained. The algorithm will be able to select from the nonterminal start symbol N_1, and a predetermined amount of generic nonterminals N_2, N_3, to N_n. the available terminals will be 'a', 'b', and 'c'. The production rules will be limited to the forms available in Chomsky Normal Form ($A \rightarrow BC$, $A \rightarrow a$).

2.3 Experiments

2.3.1 Data

The algorithm trains on strings that are generated from the target grammar through a Python script. The data for each experiment consists of 250 example strings and the number of production rule applications required to generate each string. Each dataset includes only positive examples.

2.3.2 Experiment 1

Experiment 1 aims to discover a semantic equivalent to the target grammar

$$S \rightarrow B \mid C$$
$$B \rightarrow aSb \mid ab$$
$$C \rightarrow aSc \mid ac$$

which generates the language $L(G_1) = \{(a^n w \mid w \in \{a, c\}^n,\ n > 0\}$. Examples of accepted strings are "aabb", "aabc", "aaaabccb", "aaaccc". Experiment 1 has a max string length (x) of 14 and was run with four nonterminals, including the start symbol. The purpose of Experiment 1 is to test the validity of the algorithm with a simple nondeterministic grammar.

2.3.3 Experiment 2

Experiment 2 aims to discover a semantic equivalent to the numerical based target grammar

$$S \rightarrow A \mid B \mid AB$$
$$A \rightarrow aA \mid a$$
$$B \rightarrow bBc \mid bc$$

which generates the language $L(G_2) = \{a^n b^m c^m \mid n, m \geq 0,\ n + m > 0\}$. Examples of accepted strings are "abbbccc", "bbcc", "aaaa", and "aaaaaaaabc". Experiment 2 has a max string length (x) of 30 and was run with seven nonterminals, including the start symbol. The purpose of Experiment 2 is to test whether the algorithm can find numeric limitations in structure.

2.3.4 Experiment 3

Experiment 3 aims to discover a semantic equivalent to the structurally complex target grammar

$$S \rightarrow aSa \mid bSb \mid c$$

which generates the language $L(G_3) = \{wcw^R \mid w \in \{a, b\}^n,\ n > 0\}$. Examples of the accepted strings are "abbbcbbba", "aca", "aabcbaa", and "bacab". Experiment 3 has a max string length (x) of 15 and was run with five nonterminals, including the start symbol. The purpose of Experiment 3 is to test whether the algorithm can find more complex structures.

3. ALGORITHM

3.1 Individuals

3.1.1 Representation

Each individual is represented as a 2-dimensional array with each row isomorphic to a production rule in Chomsky Normal Form. The two forms available in the genome are $[LHS,\ Nonterminal\ 1,\ Nonterminal\ 2]$ and $[LHS,\ Terminal,\ \varepsilon]$, which are equivalent to the Chomsky Normal Form rules of form $A \rightarrow BC$, $A \rightarrow a$. Additionally, each individual holds a fitness value.

3.1.2 Individual Creation

Each individual is initialized with a 'scaffolding' grammar, which ensures that all production rules are reachable and productive. Scaffolding grammars are covered in more depth in 3.5.

3.1.3 Mutation

Each individual will undergo mutation once per reproduction cycle, following crossover. In mutation, nodes may only mutate to the same type of node, meaning nonterminals may only mutate to nonterminals and terminals to terminals. Each symbol in an individual's genome has a probability of $1/(length\ of\ the\ genome)$ to mutate. If the grammar can no longer terminate after mutation, then the change is reversed and mutation is run again. After mutation, all useless rules are pruned.

3.1.4 Fitness

The fitness function is evaluated over all provided data. The fitness function heavily weights recall through the constant C_1. Recall is the ratio of generated strings that are matched from the dataset divided by the total size of the dataset and is prioritized to encourage specific matching. To prevent a short, generalized grammar that includes incorrect strings in addition to the data, a precision function is added and scaled by constant C_2. In order to build a smoother gradient across the fitness landscape, a normalized Levenshtein distance is added and averaged across the number of all data examples, then scaled by a constant C_3. The Levenshtein difference between

two strings is defined by the number of one-character edits required to make the strings match. Lastly, the number of production rules in the candidate grammar is subtracted, scaled by a small constant C_4 to discourage overfitting through hard coded terminal memorization. The function is formalized as

$$F(G) = C_1 \frac{|K|}{|E|} + C_2 \frac{|K|}{|S_{L<x}|} + \frac{C_3}{|E|} \sum_{e \in E} \max_{s \in S_G} \left(1 - \frac{Lev(e,s)}{\max\{|e|,|s|\}}\right) - C_4 |P_G|$$

where

$G = candidate\ grammar$,

$K = set\ of\ string\ matched\ from\ the\ data\ set$,

$S_{L<x} = set\ of\ strings\ generated\ by\ G\ under\ length\ x$,

$E = set\ of\ all\ strings\ in\ the\ data\ set$,

$Lev(e,s) = Levenshtein\ distance\ between\ strings\ e\ and\ s$,

$P_G = number\ of\ production\ rules\ in\ G$,

C_1, C_2, C_3, and C_4 are constants such that $C_1 > C_2 > C_3 > C_4$ and are predetermined before a trial. In this paper, $C_1 = 100$, $C_2 = 80$, $C_3 = 2$, $C_4 = 0.1$. For a given experiment, x is chosen to minimize computation while meeting the string length requirement for 250 data samples to be generated. To make the algorithm more efficient, a fitness of zero is assigned to generated strings that are longer than the corresponding example string.

3.2 Population

3.2.1 Initial Generation

The first generations are initialized with 100 randomly generated valid individuals.

3.2.2 Operating Generations and Reproduction

Each generation undergoes elitism followed by repeated selection, crossover, and mutation. The algorithm utilizes an elitism of size four. The algorithm terminates after 1000 generations.

3.3 Selection

Selection is performed with tournament selection of size three. Individuals are randomly selected to compete. The individual with the highest fitness of the tournament becomes a parent. This cycle is repeated to select a second parent. Rank-based selection was chosen over fitness-proportional to create a more structured search.

3.4 Reproduction

Each pair of parents results in four offspring. Two offspring are genetic copies and undergo crossover and mutation. The remaining two offspring are genetic copies and undergo only mutation.

3.4 Crossover

After two parents are chosen through selection, they crossover. Crossover involves copying all production rules from both parents to both offspring. First, a 'scaffolding' path is set, which guarantees that all non terminals can be reached. The second pass randomly adds one additional production rule per nonterminal to increase the expression of the grammar. If no scaffolding route is possible, crossover fails and two new parents are found through selection. After crossover, all useless rules are pruned.

3.5 Ensuring Grammar Usefulness

For each set of production rules, a valid scaffolding grammar is constructed using a bottom-up approach. Rules that produce terminals are marked as productive, and recursively traced along all dependency paths through the previous rules until at least one path reaches the start symbol. A binary production rule of form $A \rightarrow BC$ is validated once both B and C are demonstrated to be able to reach a productive rule. To prevent infinite recursion, a nonterminal may not be revisited on the same dependency path.

4. RESULTS

4.1 Experiment 1

4.1.1. Data

The target grammar and generated grammar are semantically equivalent. The target grammar is not displayed in Chomsky Normal Form for readability.

Target grammar	Generated grammar
$S \rightarrow B \mid C$	$N1 \rightarrow N2\ N4$
$B \rightarrow aSb \mid ab$	$N2 \rightarrow N3\ N1 \mid a$
$C \rightarrow aSc \mid ac$	$N3 \rightarrow a$
	$N4 \rightarrow b \mid c$

Table 1: Performance of the seed 42 trial.

Metric	**Result**
Best fitness	180.140
Recall	1.000 (250/250)
Precision	0.984
Generated strings, length ≤ 14	254

The algorithm ran on initialization Seed42 scored an global optimum fitness of 180.140, a perfect recall on 250 examples, and a precision of 0.984 over 254 generated strings of at most length 14.

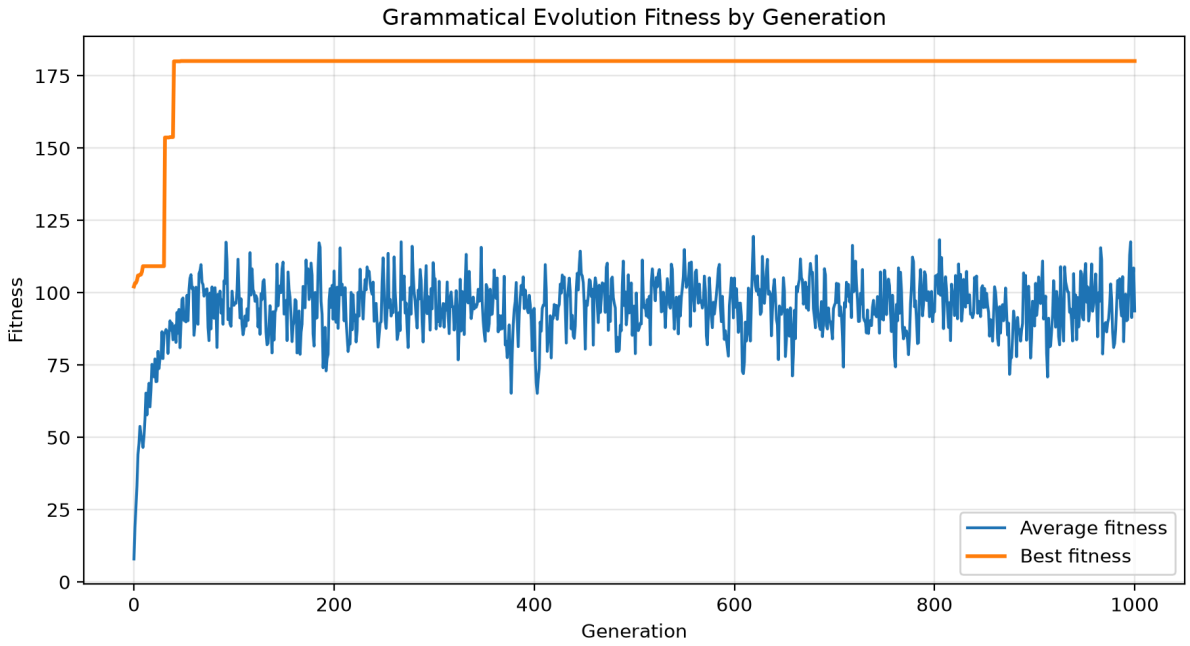


Figure 1. The average and best fitness of the Seed42 run of Experiment 1 over 1000 generations. The algorithm reached one of the fitness landscape's global optima by generation 50. The population's average fitness fluctuates around 96.298 after a rapid increase from generations 0 to 50.

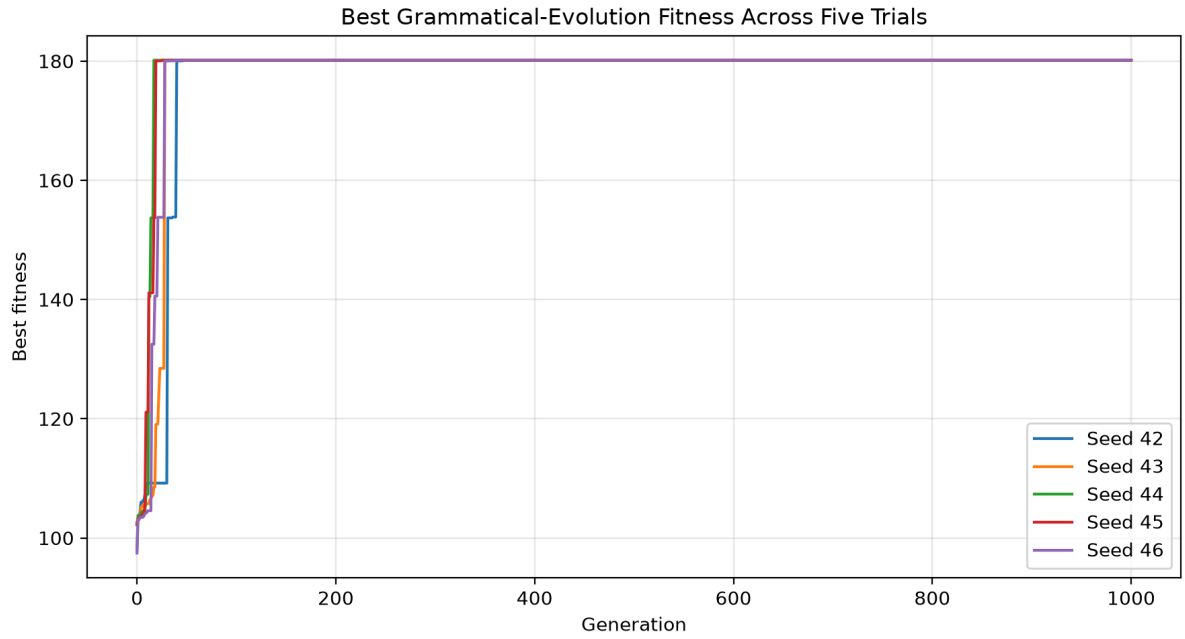


Figure 2. The best fitness across six trials of Experiment 1 with seeds 42 through 46 over 1000 generations. Each trial reached a global optimum by generation 50 and could not improve further.

4.1.2 Analysis

Given a maximum string length of 14, the target grammar generates 254 strings, only 250 of which were in the dataset. Thus the precision is $250/254 \approx 0.984$. The remaining four strings are valid members of the target language, but treated as false positives as they are excluded from the data.

The rapid increase of the Seed 42 population's average fitness between generations 0 and 50 suggests the algorithm generated at least one grammar that utilized the recall term of the fitness function. The smoothness of the increase of the population's average fitness demonstrates that the normalized Levenshtein distance term provided enough guidance towards matched cases in early generations. Overall, the algorithm performed well in Experiment 1.

4.2 Experiment 2

The target grammar and generated grammar are semantically equivalent. The target grammar is not displayed in Chomsky Normal Form for readability.

Target Grammar	Generated Grammar
$S \rightarrow A \mid B \mid AB$ $A \rightarrow aA \mid a$ $B \rightarrow bBc \mid bc$	$N1 \rightarrow N4\ N6 \mid N7\ N1 \mid a$ $N3 \rightarrow c$ $N4 \rightarrow b$ $N5 \rightarrow N4\ N6$ $N6 \rightarrow N5\ N3 \mid c$ $N7 \rightarrow a$

Table 2: Performance of the seed 42 trial.

Metric	**Result**
Best fitness	179.531
Recall	1.000 (250/250)
Precision	0.980
Generated strings, length ≤ 30	255

The algorithm ran on initialization Seed42 scored an global optimum fitness of 179.531, a perfect recall on 250 examples, and a precision of 0.980 over 255 generated strings of at most length 30.

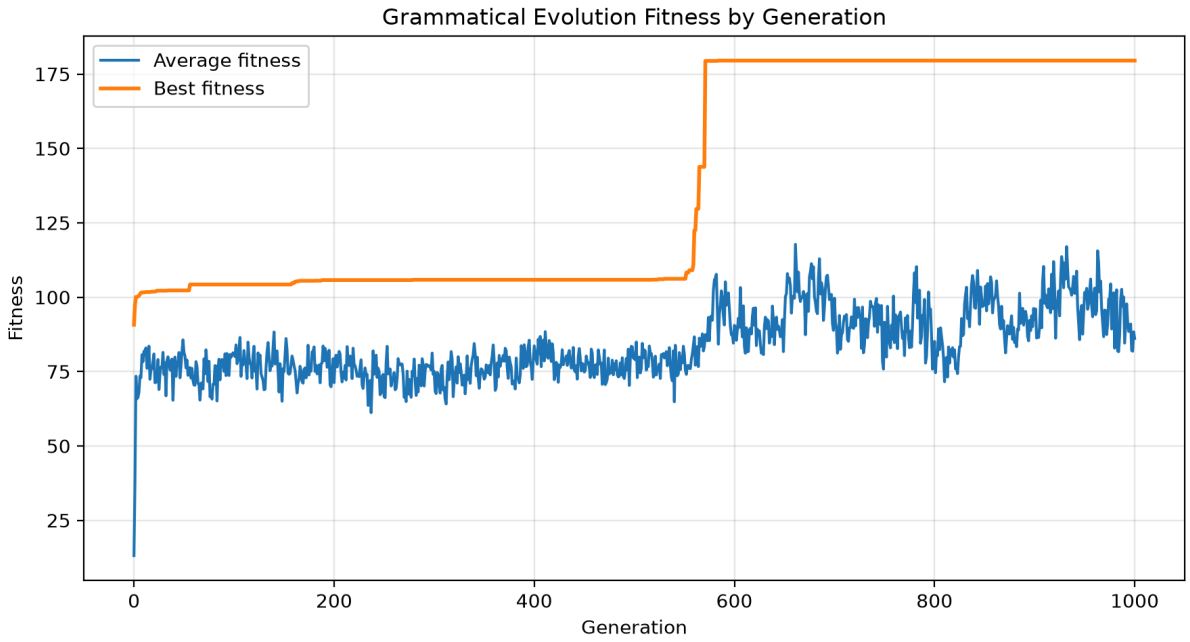


Figure 3. The average and best fitness of the Seed42 run of Experiment 2 over 1000 generations. The population's best fitness stabilized under 106.186 before reaching a global optimum by generation 600. The population's average fitness shows a sharp rise in very early generations stabilizing to fluctuations around 73.663 from generations 0 to 575. From generation 600 to 1000, the average fitness fluctuates around 92.556, an approximate increase of 25.65%.

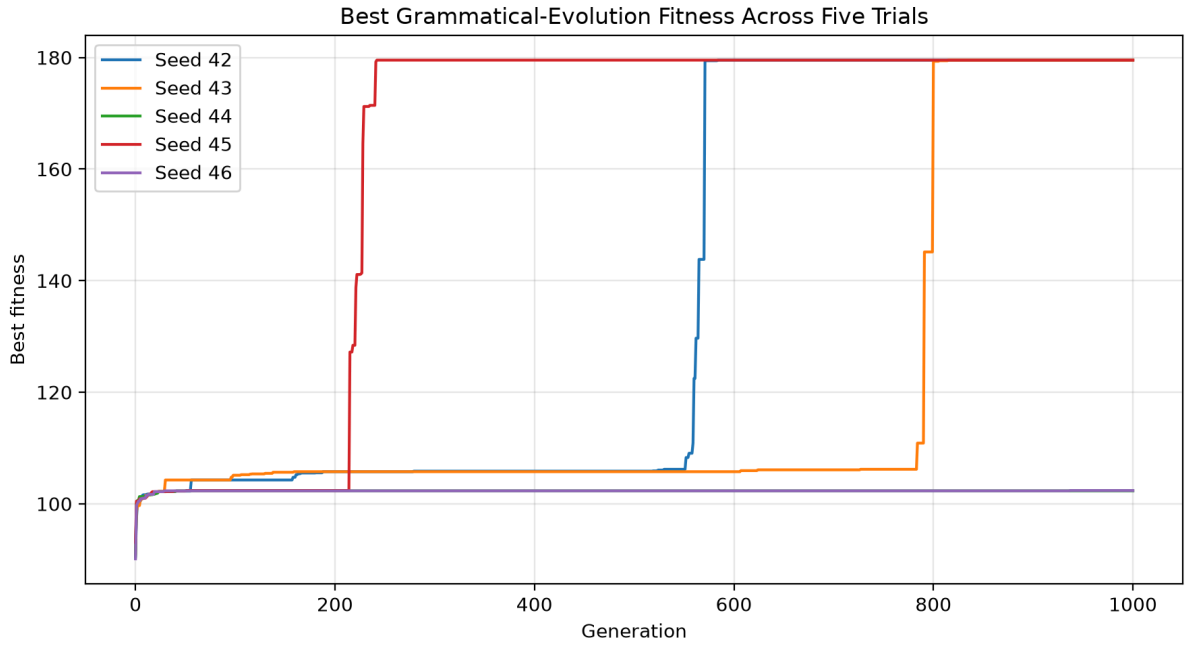


Figure 4. The best fitness across six trials of Experiment 2 with seeds 42 through 46 over 1000 generations. Four out of five of the trials reached a global optimum by generation 825. Seed 45 reached a global optimum before generation 250, the fastest trial to do so. Seeds 42 and 44

progressed at nearly an identical rate, reaching a global optimum before generation 600. Seed 43 reached a global optimum before generation 825. Seed 46 began with the lowest initial best fitness of the trials, and reached a fitness plateau by generation 75.

4.2.2 Analysis

Given a maximum string length of 30, the target grammar generates 255 strings, only 250 of which were in the dataset. Thus the precision is $250/255 \approx 0.980$. The remaining five strings are valid members of the target language, but treated as false positives as they are excluded from the data.

Additionally, every trial reached a fitness plateau between 100 and 110 fitness for at least 175 generations, indicating a common trend. Each trial that reached a global optimum did so through a sharp rise in fitness of more than 70. This can be explained by an increase in precision from approximately 0.05 to 0.98, resulting in a fitness increase of $80(0.98 - 0.05) = 74.4$. This suggests Experiment 2 initially discovered a broad grammar with high recall and low precision. Evolution later found more restrictive grammars with both high recall and high precision.

Lastly, Seed 46 began with the lowest best fitness, but reached a fitness plateau around the same generation count as the other trials. This is consistent with the normalized Levenshtein term of the fitness function creating a gradient across the early generations of the fitness landscape.

4.3 Experiment 3

4.3.3 Data

The target grammar and generated grammar are not semantically equivalent. The target grammar is not displayed in Chomsky Normal Form for readability.

Target Grammar	Generated Grammar
$S \rightarrow aSa \mid bSb \mid c$	$N1 \rightarrow N5\ N4 \mid c$ $N4 \rightarrow N1\ N5$ $N5 \rightarrow a \mid b$

Table 3: Performance of the seed 42 trial.

Metric	**Result**
Best fitness	107.360
Recall	1.000 (250/250)
Precision	0.073
Generated strings, length ≤ 15	3413

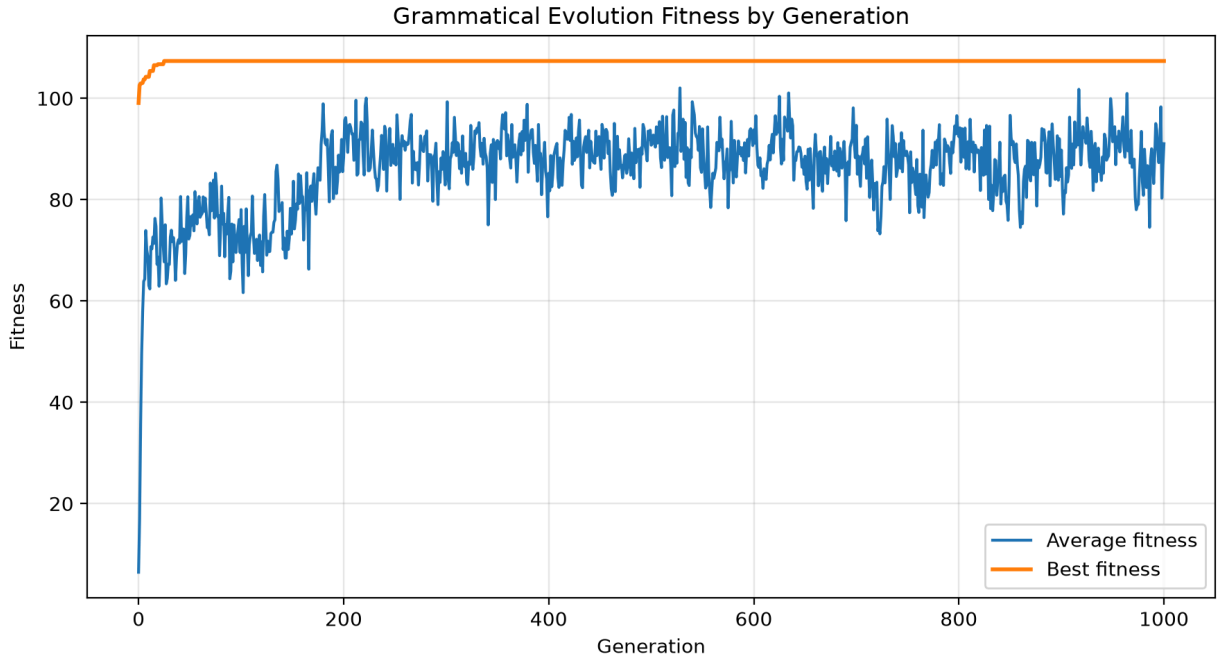


Figure 5. The average and best fitness of the Seed42 run of Experiment 3 over 1000 generations. The population's best fitness modestly increased before generation 25, and faced no improvement over the rest of the trial. The max best fitness reached was 107.360. The population's average fitness rapidly increased before generation 25, and continued to climb until generation 175, where it fluctuated around an average fitness of 89.590.

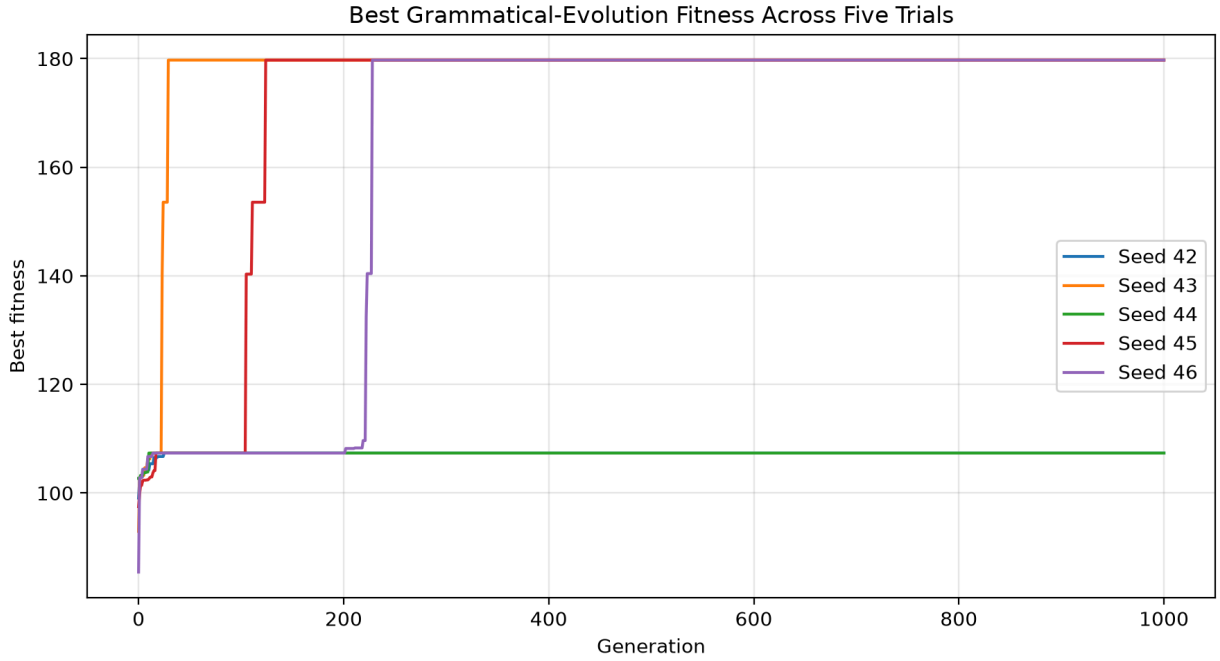


Figure 6. The best fitness across six trials of Experiment 3 with seeds 42 through 46 over 1000 generations. Three of the five trials reached a global optimum before generation 250. Two trials became stuck on local optimum, both of which reached a plateaued fitness of 107.360.

4.3.2 Analysis

Seed 42 reached a nonoptimal fitness plateau, indicating it got trapped on local optimum. Seed 44 was trapped on similarly structured local optima. This, in addition to a shared low precision value of 0.073 and high recall of 1.00, indicates that both trials were stuck on an overgeneralized grammar.

However, in the additional trials, three out of five trials reached a global optimum with a semantically equivalent grammar. Both Seed 42 and Seed 44, the failed trials, had only five production rules, while the other three trials had seven. Given the complexity of the grammar, the algorithm performed well on Experiment 3, but the failed trials demonstrate the sensitivity to initialization and potentially to prematurely converge on an overgeneralized trial.

5. CONCLUSION

Across all three experiments, the algorithm reached a semantic equivalent to the target grammar in at least three trials. In Experiment 1, all trials reached a semantic equivalent, and zero trials reached an overgeneralization fitness plateau. In Experiment 2, four of five trials reached a semantic equivalent, and every grammar reached an overgeneralization fitness plateau for at least 200 generations. In Experiment 3, three out of five trials reached a semantic equivalent, and two out of five trials reached an overgeneralization plateau. These results indicate that the algorithm can successfully discover complex grammar representations, but is hindered by local optima created by broad grammars. Allowing the algorithm to use more nonterminals than the optimal grammar requires may allow the algorithm to escape the local optima, but requires further validation. Additionally, allowing the mutation of production rules may provide the algorithm a method to more efficiently explore the search space and to ignore local optima. This approach also requires further testing.

The precision function encourages more restrictive grammars, but is limited by the size of the dataset relative to allowed string length. Thus, the precision function may overfit grammars. This was not demonstrated in any experiments, it is a potential limitation.

The most essential future work should evaluate methods for escaping overgeneralization based local optima, but should additionally test the algorithm on datasets where the longest string is longer than allowed by the algorithm and investigate whether increasing the amount of available nonterminals increases consistency.

6. APPENDICES

6.1 Experiment 1 Seed 42 Data

Generation 0: average = 7.949172, best = 102.185324
Generation 25: average = 73.776871, best = 109.198229

Generation 50: average = 90.404975, best = 180.140157
Generation 75: average = 83.458395, best = 180.140157
Generation 100: average = 95.666970, best = 180.140157
Generation 125: average = 94.469911, best = 180.140157
Generation 150: average = 92.628748, best = 180.140157
Generation 175: average = 97.651298, best = 180.140157
Generation 200: average = 107.541787, best = 180.140157
Generation 225: average = 91.966290, best = 180.140157
Generation 250: average = 101.280037, best = 180.140157
Generation 275: average = 93.842387, best = 180.140157
Generation 300: average = 104.825785, best = 180.140157
Generation 325: average = 87.011648, best = 180.140157
Generation 350: average = 86.067178, best = 180.140157
Generation 375: average = 88.904417, best = 180.140157
Generation 400: average = 94.617486, best = 180.140157
Generation 425: average = 95.966866, best = 180.140157
Generation 450: average = 95.635047, best = 180.140157
Generation 475: average = 98.671910, best = 180.140157
Generation 500: average = 95.417596, best = 180.140157
Generation 525: average = 108.034117, best = 180.140157
Generation 550: average = 114.953099, best = 180.140157
Generation 575: average = 105.199407, best = 180.140157
Generation 600: average = 105.120875, best = 180.140157
Generation 625: average = 101.179265, best = 180.140157
Generation 650: average = 100.672984, best = 180.140157
Generation 675: average = 105.374194, best = 180.140157
Generation 700: average = 95.713271, best = 180.140157
Generation 725: average = 90.151529, best = 180.140157
Generation 750: average = 107.824972, best = 180.140157
Generation 775: average = 85.073164, best = 180.140157
Generation 800: average = 93.414525, best = 180.140157
Generation 825: average = 101.538443, best = 180.140157
Generation 850: average = 101.474583, best = 180.140157
Generation 875: average = 71.798413, best = 180.140157
Generation 900: average = 88.517582, best = 180.140157
Generation 925: average = 82.011188, best = 180.140157
Generation 950: average = 106.169201, best = 180.140157
Generation 975: average = 95.784275, best = 180.140157
Generation 1000: average = 93.706155, best = 180.140157

6.2 Experiment 2 Seed 42 Data

Generation 0: average = 13.136972, best = 90.745338
Generation 25: average = 68.955361, best = 102.190389
Generation 50: average = 81.407312, best = 102.274241
Generation 75: average = 78.448526, best = 104.267985
Generation 100: average = 80.530799, best = 104.267985
Generation 125: average = 78.093351, best = 104.267985
Generation 150: average = 75.002069, best = 104.267985
Generation 175: average = 73.886091, best = 105.519890
Generation 200: average = 71.253094, best = 105.755547
Generation 225: average = 73.298663, best = 105.755547
Generation 250: average = 66.552923, best = 105.755547
Generation 275: average = 72.366315, best = 105.755547
Generation 300: average = 75.061685, best = 105.855547
Generation 325: average = 74.676266, best = 105.855547
Generation 350: average = 72.449427, best = 105.855547
Generation 375: average = 79.892296, best = 105.855547
Generation 400: average = 83.642304, best = 105.855547
Generation 425: average = 77.209497, best = 105.855547
Generation 450: average = 74.831401, best = 105.855547
Generation 475: average = 74.145933, best = 105.855547
Generation 500: average = 79.289964, best = 105.855547
Generation 525: average = 77.561090, best = 106.044735
Generation 550: average = 80.857691, best = 106.186480
Generation 575: average = 85.383747, best = 179.431373
Generation 600: average = 90.546664, best = 179.531373
Generation 625: average = 88.675285, best = 179.531373
Generation 650: average = 92.046850, best = 179.531373
Generation 675: average = 106.851098, best = 179.531373
Generation 700: average = 99.688097, best = 179.531373
Generation 725: average = 94.199869, best = 179.531373
Generation 750: average = 91.579945, best = 179.531373
Generation 775: average = 83.012347, best = 179.531373
Generation 800: average = 85.153363, best = 179.531373
Generation 825: average = 83.365930, best = 179.531373
Generation 850: average = 97.312981, best = 179.531373
Generation 875: average = 87.594019, best = 179.531373
Generation 900: average = 92.603460, best = 179.531373
Generation 925: average = 93.715729, best = 179.531373
Generation 950: average = 101.618618, best = 179.531373

Generation 975: average = 99.367809, best = 179.531373
Generation 1000: average = 86.124176, best = 179.531373

6.3 Experiment 3 Seed 42 Data

Generation 0: average = 6.457986, best = 99.076537
Generation 25: average = 67.711674, best = 107.359947
Generation 50: average = 72.910667, best = 107.359947
Generation 75: average = 85.212979, best = 107.359947
Generation 100: average = 78.026584, best = 107.359947
Generation 125: average = 69.032907, best = 107.359947
Generation 150: average = 75.455775, best = 107.359947
Generation 175: average = 86.294110, best = 107.359947
Generation 200: average = 88.309545, best = 107.359947
Generation 225: average = 85.713451, best = 107.359947
Generation 250: average = 90.062271, best = 107.359947
Generation 275: average = 92.576852, best = 107.359947
Generation 300: average = 88.153028, best = 107.359947
Generation 325: average = 88.713305, best = 107.359947
Generation 350: average = 91.666273, best = 107.359947
Generation 375: average = 89.642930, best = 107.359947
Generation 400: average = 85.203539, best = 107.359947
Generation 425: average = 87.732962, best = 107.359947
Generation 450: average = 88.566183, best = 107.359947
Generation 475: average = 88.886404, best = 107.359947
Generation 500: average = 90.736458, best = 107.359947
Generation 525: average = 87.186583, best = 107.359947
Generation 550: average = 93.976013, best = 107.359947
Generation 575: average = 78.384180, best = 107.359947
Generation 600: average = 93.768945, best = 107.359947
Generation 625: average = 100.392671, best = 107.359947
Generation 650: average = 88.578364, best = 107.359947
Generation 675: average = 92.480365, best = 107.359947
Generation 700: average = 94.665939, best = 107.359947
Generation 725: average = 82.621791, best = 107.359947
Generation 750: average = 88.552733, best = 107.359947
Generation 775: average = 86.336992, best = 107.359947
Generation 800: average = 92.422374, best = 107.359947
Generation 825: average = 93.747476, best = 107.359947
Generation 850: average = 96.631873, best = 107.359947
Generation 875: average = 86.658971, best = 107.359947
Generation 900: average = 89.778508, best = 107.359947

Generation 925: average = 96.122651, best = 107.359947
Generation 950: average = 91.424385, best = 107.359947
Generation 975: average = 79.053055, best = 107.359947
Generation 1000: average = 91.033784, best = 107.359947